

Edge cases (a task that needs no tool at all) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `run_agent()` from `build_react_loop`'s Advanced Version (returns an `AgentResult` with `final_answer` and `steps`), with `get_weather` and `celsius_to_fahrenheit` as the only 2 registered tools.

Basic Version

```
# no_tool_needed_practice.py
result = run_agent("What's the capital of France?", max_iterations=5)
print(result.final_answer)
print(result.steps)
```

Expected output:

```
The capital of France is Paris.
[]
```

`result.steps` comes back empty because the model's very first response already had no `tool_calls` — it answered directly. This works correctly and confirms the exercise's core claim, but it's something you read once, not something a test suite can check for you automatically.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — one real assertion

```
# no_tool_needed_practice.py
def test_no_tool_call_for_general_knowledge():
    result = run_agent("What's the capital of France?", max_iterations=5)
    assert len(result.steps) == 0
    assert "Paris" in result.final_answer
```

Expected output when run with pytest:

1 passed

Approach 2 — a small list of clearly-no-tool prompts

```
# no_tool_needed_practice.py
NO_TOOL_PROMPTS = [
    "What's the capital of France?",
    "What year did World War II end?",
    "Who wrote Romeo and Juliet?",
]

def test_clear_prompts_never_call_a_tool():
    for prompt in NO_TOOL_PROMPTS:
        result = run_agent(prompt, max_iterations=5)
        assert len(result.steps) == 0, f"unexpected tool call for: {prompt}"
```

Expected output when run with pytest:

1 passed

Difference from Basic: Basic reads printed output once and trusts it looks right. Approach 1 turns that same check into a real `assert`, so it can run in CI and fail loudly if it ever stops being true. Approach 2 widens the check to several prompts in one test, catching a case where the model

behaves correctly on one question but not another — a single example was never proof of the general claim, just one data point toward it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a genuinely borderline prompt, observed rather than forced

```
# no_tool_needed_practice.py
def test_borderline_prompt_is_observed_not_forced():
    result = run_agent("Is Paris generally a warm city?", max_iterations=5)
    print("tool calls made:", len(result.steps))
    print("answer:", result.final_answer)
    # deliberately no strict assertion on step count here: this prompt sits
    # close enough to get_weather's purpose that a reasonable model could
    # go either way, and forcing an assertion would make this test flaky
    # through no fault of the agent loop itself
    assert result.final_answer # the one thing that should always hold: it answered
```

Expected output (one possible run):

```
tool calls made: 0
answer: Yes, Paris generally has a temperate climate with mild summers.
```

Approach 2 — catching a vague tool description on purpose

```
# no_tool_needed_practice.py
from unittest.mock import patch

def test_vague_tool_description_can_cause_unnecessary_calls():
    """Demonstrates the Advanced Hint's warning: a badly-worded tool
    description can pull in a tool call that a precise one wouldn't."""
    vague_schema = {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "get information about a city", # too vague on purpose
            "parameters": {"type": "object", "properties": {"city": {"type": "string"}},
        },
    }
    with patch("__main__.weather_tool_schema", vague_schema):
```

```
result = run_agent("What's the capital of France?", max_iterations=5)
print("tool calls with vague description:", len(result.steps))
```

Expected behavior: with the real, specific description ("get the current weather for a city"), this prompt reliably makes zero tool calls (Approach 1 above, and the Intermediate tests). With the deliberately vague one above, it's meaningfully more likely to call `get_weather` anyway — not guaranteed every single run, but noticeably more often, because "get information about a city" doesn't rule out "capital city" as a match the way a precise description does. Running this side by side with a real tool description is the clearest possible demonstration that this exercise is really testing your Doc06 tool *descriptions*, not your agent loop's code.

Difference from Intermediate: Intermediate proves the safe case reliably, with real assertions. Approach 1 adds a genuinely ambiguous prompt and treats it correctly — observed and printed, not asserted, because the model's exact behavior there isn't something your loop's correctness controls. Approach 2 goes one step further and actively demonstrates the *cause* of an unwanted tool call: not a bug in `run_agent()` at all, but a vague tool description upstream in Doc06.

Which one should you actually write? Intermediate Approach 2's kind of test — a short list of clearly-no-tool prompts, each asserted to produce zero tool calls — is exactly what belongs in `test_agent.py` for the Build Task, matching the Test Cases table's "Task needs no tool → Direct answer, zero tool calls" row. Keep Advanced Approach 1's kind of borderline-prompt check around as a debugging tool, not a hard assertion — and if you ever see it start reliably calling a tool it shouldn't, go fix the tool's *description* first, per Approach 2, before assuming your loop is broken.